OUTREACH-HUB PLATFORM

Complete Process & Execution Workflow Guide

How Frontend, Backend, PDF Parser, SQLite Database & Gmail SMTP Work Together

System Architect: Vishal Yadav	**Stack:** FastAPI (Python 3.14) + React 18 + Vite
Target Dataset: 1,842 Contacts (21 Pages PDF)	**Safety Model:** 100% Local / Zero Cloud AI Leaks
Outbound Channel: Gmail SMTP STARTTLS (Port 587)	**Audit Storage:** SQLite Database (`email_sender.db`)

```
================================================================================================== MASTER SYSTEM
WORKFLOW PIPELINE ================================================================================================== [
PHASE 1: BOOT & HEALTH CHECK ] FastAPI starts up -> verifies SQLite tables -> Vite starts React dashboard -> Navbar
begins 10-second heartbeat check (GET /health) -> Status turns GREEN. | v [ PHASE 2: INGESTION & NORMALIZATION ]
pdfplumber reads contactInfo.pdf (21 pages) -> extracts 1,842 rows -> clean_email_str fixes wrapped emails -> contacts
cached in memory (0.2ms lookups). | v [ PHASE 3: PREVIEW & DUPLICATE INSPECTION ] User selects SNo range (e.g. 1-10) ->
extract_first_name strips honorifics (Mr., Dr.) -> Jinja templates render subject & body -> SQLite checks if already
emailed. | v [ PHASE 4: CAMPAIGN DISPATCH (SAFETY GUARDED) ] User clicks "Send" -> 2-Step modal requires confirmation ->
POST /send receives confirm:true -> Gmail STARTTLS connects -> For each contact: Check SQLite Duplicate -> Send MIME +
Resume.pdf -> Log 'SENT' in SQLite -> Wait 5.0s rate limit pause. | v [ PHASE 5: AUDIT, ANALYTICS & UI REFRESH ]
Execution summary returned -> React state updates -> Dashboard KPI cards increment -> Recharts charts refresh -> Logs
table shows persistent SQLite audit records.
==================================================================================================
```

Guide Navigation (10 Core Workflow Modules)

Phase	Module & Workflow Focus	Page
1	System Boot, Initialization & Real-Time Heartbeat Workflow	2
2	PDF Contact Extraction & Multi-Line Cleaning Workflow	3
3	Personalization & First-Name Decision Tree Workflow	4
4	Core Campaign Execution Engine: The Decision Loop Flowchart	5
5	Concrete Step-by-Step Walkthrough: Tracing Real Contacts	6
6	The 3-Tier Safety Shield & Dry-Run Simulation Workflow	7
7	Frontend Reactive State & UI Data Flow (React 18 + Vite)	8
8	SQLite State Machine & Transactional Audit Trail Workflow	9
9	Headless CLI Execution Workflow & Operational Diagnostics	10

1. System Boot, Initialization & Real-Time Heartbeat Workflow

Understanding what happens when Outreach-Hub powers on is essential. The platform is designed to be completely self-healing and zero-configuration: when you boot the server, all necessary database tables and directory paths are created automatically before any user requests arrive.

```
+-------------------------------------------------------------------------------------------------+ | SYSTEM STARTUP
SEQUENCE FLOW | +-------------------------------------------------------------------------------------------------------+
1. BACKEND INITIALIZATION (uvicorn app.main:app --port 8000) +----------------------+ +----------------------+
+----------------------+ | FastAPI Server Starts | --> | Lifespan Startup Hook | --> | init_db() Execution | | Loads
app.config | | @asynccontextmanager | | Checks email_logs tab | +----------------------+ +----------------------+
+----------------------+ | v +----------------------+ | CORS Middleware Setup | | Allows :5173 origin |
+----------------------+ 2. FRONTEND INITIALIZATION (npm run dev on port 5173) +----------------------+
+----------------------+ +----------------------+ | Vite Server Starts | --> | React App Mounts | --> | AppLayout Shell
Loads | | Hot Module Reloading | | BrowserRouter mounts | | Sidebar + Navbar View | +----------------------+
+----------------------+ +----------------------+ | v 3. LIVE HEARTBEAT HANDSHAKE (Every 10 Seconds) |
+-------------------------------------------------------------------------+ | Navbar.jsx fires background ping: Axios GET
http://127.0.0.1:8000/health | --> Backend verifies: Database connected? Contact PDF found? Resume.pdf found? SMTP set? |
--> Backend returns: {"status": "healthy", "database": "connected", "total_contacts": 1842} | --> Frontend Navbar lights
up glowing GREEN badge: "System Online & Operational"
+-------------------------------------------------------------------------------------------------+
```

What Happens if a Subsystem Fails at Boot?

Database: SQLite database (`data/email_sender.db`) is automatically generated if missing. No manual migrations needed.
Missing Resume.pdf: The `/health` check flags `resume_pdf.exists = False`. The UI turns yellow and prevents real email dispatching until `Resume.pdf` is placed in `backend/`.
Backend Offline: If FastAPI is stopped, the Navbar heartbeat fails gracefully and displays a red badge: *'Backend Offline'*.

Dependency Injection Per Request (`get_db`)

Every incoming HTTP call to `/health`, `/contacts`, `/preview`, `/send`, or `/logs` injects an independent SQLAlchemy database session via `db: Session = Depends(get_db)`. When the request completes, FastAPI guarantees the session is closed in a `finally` block, eliminating SQLite database locks.

2. PDF Contact Extraction & Multi-Line Cleaning Workflow

Outreach-Hub extracts candidate recruiter directories directly from `contactInfo.pdf` (21 pages, 1,842 contacts). Because PDF files contain arbitrary spatial layout rather than structured database records, the parser employs a resilient multi-step cleaning and validation pipeline.

```
+-------------------------------------------------------------------------------------------------+ | PDF EXTRACTION PIPELINE FLOW | +-------------------------------------------------------------------------------------------------+
1. FILE INGESTION: Open 'contactInfo.pdf' via pdfplumber Reads 21 pages sequentially. Check file modification time (mtime). | v 2. TABLE STRUCTURE EXTRACTION: page.extract_tables() Detects 5-column table: [ SNo | Name | Email | Title | Company ] Detects and strips table header rows ("SNo", "Sl No", "#"). | v 3. THE MULTI-LINE EMAIL WRAPPING REPAIR: clean_email_str() Problem: Dense PDF columns wrap long emails across multiple lines: "gitanjali.venkatesh@excelenciaconsulting.\ncom" Solution: Aggressively strip '\r', '\n', and whitespace ' ': --> Becomes: "gitanjali.venkatesh@excelenciaconsulting.com" (100% Valid!) | v 4. FALLBACK REGEX ANCHOR PARSER (Safety Net) If table borders are missing on any page: Scans text using EMAIL_ANCHOR_REGEX: ([a-zA-Z0-9_.+-]+@...\.[a-zA-Z]{2,}) Uses detected email position as delimiter to reconstruct SNo, Name, Title, Company. | v 5. EMAIL SYNTAX VALIDATION: validate_email() Validates RFC compliance. Malformed records are diverted to skipped audit array. | v 6. IN-MEMORY CACHING LAYER (_CACHED_CONTACTS) First Call: 21 pages parsed in ~10 seconds. All Subsequent Calls: Served from memory in 0.2 milliseconds! +-------------------------------------------------------------------------------------------------+
```

Sample Extracted Record Transformation

Field	Raw PDF Content (Messy / Wrapped)	Normalized Output (Clean Record)
SNo	`" 394 "`	394 (Integer)
Name	`"Gitanjali Venkatesh\n"`	"Gitanjali Venkatesh"
Email	`"gitanjali.venkatesh@excelenciaconsulting.\ncom"`	**"gitanjali.venkatesh@excelenciaconsulting.com"**
Title	`"HR - Talent Acquisition"`	"HR - Talent Acquisition"
Company	`"Excelencia Consulting\r\n"`	"Excelencia Consulting"

3. Personalization & First-Name Decision Tree Workflow

Generic cold emails are immediately ignored. Outreach-Hub uses a deterministic personalization engine (`app/email/template.py`) that extracts friendly, natural first names from raw corporate directory listings, handling honorifics, multi-word names, and single-letter initials.

```
+-------------------------------------------------------------------------------------------------+ | FIRST-NAME
EXTRACTION DECISION TREE |
+-------------------------------------------------------------------------------------------------+ [ Input: raw_name
string ] | v Is raw_name empty? / \ (YES)/ \(NO) v v [ "Hiring Team" ] Split into whitespace tokens | v Does token[0]
match HONORIFICS? ("Mr.", "Ms.", "Mrs.", "Dr.", "Prof.") / \ (YES)/ \(NO) v v Strip matching honorifics Keep initial
token (e.g. "Dr. Rajesh" -> "Rajesh") | | | +------------------+--------------------+ | v Is token a single-letter
initial? (e.g. "A." or "A. K. Sharma") / \ (YES)/ \(NO) v v Keep full cleaned name Isolate first name & Capitalize ("A.
K. Sharma" -> "A. K. Sharma") (e.g. "akanksha puri" -> "Akanksha") Prevents awkward "Hi A," Natural greeting: "Hi
Akanksha," +-------------------------------------------------------------------------------------------------+
```

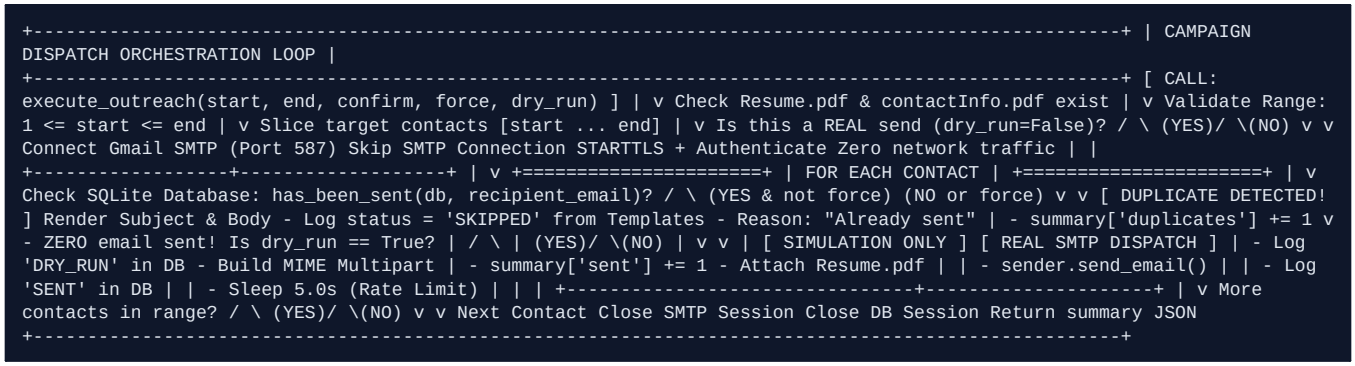
The Jinja-Style Substitution Flow (`render_email`)

1. Loads `backend/templates/subject.txt` and `email.txt` from disk.
2. Assembles context dictionary combining recipient metadata with sender info from `app/config.py`.
3. Executes case-insensitive regex substitution replacing every `{{Key}}` or `{{ Key }}` token.
4. Returns fully rendered `(subject, body)` tuple ready for preview or SMTP dispatch.

Template Placeholder	Context Extraction Rule	Rendered Result Example
`Hi {{Name}},`	First name after honorific stripping	**"Hi Akanksha,"**
`opportunities at {{Company}}`	Normalized company field	"opportunities at SourceFuse Technologies"
`matching role for {{Title}}`	Normalized title field	"matching role for Talent Acquisition"
`Best regards, {{SenderName}}`	Configured SENDER_NAME	"Best regards, Vishal Yadav"
`LinkedIn: {{SenderLinkedIn}}`	Configured SENDER_LINKEDIN	"linkedin.com/in/vishal-yadav21"

4. Core Campaign Execution Engine: The Decision Loop Flowchart

The centerpiece of Outreach-Hub is the orchestration service (`app/services/outreach.py`). It coordinates contact slicing, duplicate detection, dry-run simulation, SMTP socket communication, and SQLite audit logging.



5. Concrete Step-by-Step Walkthrough: Tracing Real Contacts

To visualize exactly how the code executes, let's trace a live batch targeting SNo 1, 2, and 3 from `contactInfo.pdf`.

Scenario Setup

User Input: Start = 1, End = 3, Dry Run = False, Force = False, Delay = 5.0s, Confirm = True.
Historical Database State: SNo 2 (`akanksha.sogani@perennialsys.com`) was previously emailed last week (status = 'SENT'). SNo 1 and SNo 3 have never been contacted.

Step & Contact	Engine Actions & Code Path	Outcome & Database Log
Step 1 **Pre-Flight**	• Verifies Resume.pdf and contactInfo.pdf exist on disk. • Connects to smtp.gmail.com:587 via TLS. • Authenticates with Google App Password.	SMTP connection established. Batch ready to begin.
Step 2 **SNo 1: Akanksha Puri** **SourceFuse Tech**	1. Query SQLite: has_been_sent('akanksha.puri@sourcefuse.com') → **False**. 2. extract_first_name() → 'Akanksha'. 3. Render template: *'Hi Akanksha... opportunities at SourceFuse Technologies...'* 4. Construct MIME message & attach Resume.pdf (124 KB). 5. SMTP transmits message to Gmail server (Response: 250 OK). 6. Engine pauses execution for **5.0 seconds** (Rate Limiter).	**STATUS: SENT** Record written to SQLite table email_logs with UTC timestamp. summary['sent'] += 1
Step 3 **SNo 2: Akanksha Sogani** **Perennial Systems**	1. Query SQLite: has_been_sent('akanksha.sogani@perennialsys.com') → **True!** 2. Evaluates force == False. 3. **DUPLICATE DETECTED!** Dispatch is aborted immediately. 4. Zero SMTP communication occurs. No email is sent. 5. Rate limit delay is bypassed (no network activity).	**STATUS: SKIPPED** Audit record written to SQLite with reason: *'Email already sent previously'*. summary['duplicates'] += 1
Step 4 **SNo 3: iB Hubs Recruiter** **iB Hubs**	1. Query SQLite: has_been_sent(...) → **False**. 2. Renders personalized email for iB Hubs. 3. Constructs MIME container with attached Resume.pdf. 4. SMTP transmits message (Response: 250 OK).	**STATUS: SENT** Record written to SQLite. summary['sent'] += 1
Step 5 **Teardown & Summary**	• SMTP connection closed cleanly via smtp.quit(). • Scoped database session closed via db.close(). • Returns summary JSON to caller.	**Final Summary:** **Processed: 3 | Sent: 2** **Duplicates Blocked: 1 | Failed: 0**

6. The 3-Tier Safety Shield & Dry-Run Simulation Workflow

Accidental mass emailing is a critical failure mode in outreach automation. Outreach-Hub implements a 3-tier defensive architecture ensuring that real emails are NEVER sent without deliberate, double-confirmed human intent.

```
+-------------------------------------------------------------------------------------------------+ | THE 3-TIER
SAFETY SHIELD ARCHITECTURE |
+-------------------------------------------------------------------------------------------------+ [ TIER 1:
FRONTEND SAFETY MODAL (React 18) ] Clicking "Start Campaign" does NOT send emails. It pops up an accessible confirmation
modal: "You are about to send 10 emails. Sender: vishalyadav.it26@gmail.com. Delay: 5.0s." The user must explicitly click
"Confirm & Send" to proceed. | v [ TIER 2: BACKEND CONFIRM GUARD (FastAPI routes.py) ] if not req.dry_run and not
req.confirm: raise HTTPException(status_code=400, detail="Safety guard violation: confirm must be true.") Even if an
external script or curl probes POST /send, it is rejected unless confirm: true is passed. | v [ TIER 3: INTERACTIVE CLI
PROMPT (cli.py) ] When running via terminal, the CLI halts and asks: "You are about to send X emails using Gmail SMTP.
Continue? [y/N]: " Pressing Enter or typing anything other than 'y' or 'yes' immediately terminates the process.
+-------------------------------------------------------------------------------------------------+
```

Dry-Run Simulation Mode: Zero Risk Testing

Dry-Run Mode (`dry_run: true`) is a full behavioral simulation sandbox:

- **Full Ingestion & Slicing:** Extracts real contacts from the PDF exactly like a production run.
- **Template Personalization:** Renders every subject and email body and verifies variable replacements.
- **Duplicate Detection Test:** Runs live queries against SQLite to flag contacts that were previously emailed.
- **Zero SMTP Activity:** Does NOT connect to Gmail SMTP. Zero network packets leave your computer.
- **Audit Trail Recording:** Records the simulation in SQLite with status DRY_RUN for verification.

The Force Override Flag (`force: true`)

In rare cases where you intentionally need to re-email a recruiter (e.g. sending an updated resume after several months), passing `force: true` bypasses the duplicate rejection guard while preserving a permanent audit record of the re-send.

7. Frontend Reactive State & UI Data Flow (React 18 + Vite)

The React 18 user interface is organized into distinct stateful views that communicate with the FastAPI backend through an asynchronous service layer (`src/services/api.js`).

```
+-------------------------------------------------------------------------------------------------+ | REACT COMPONENT INTERACTION & STATE FLOW |
+-------------------------------------------------------------------------------------------------+ [ AppLayout.jsx ] (Master Shell) |-- Sidebar.jsx (Navigation: Dashboard, Contacts, Campaign, Preview, Logs, Diagnostics) |-- Navbar.jsx (Heartbeat ping -> GET /health every 10s -> Live green/red status badge) | +---> [ Dashboard.jsx ] | Fetches: logsService.getLogs(), healthService.checkHealth() | Computes: Total Processed, Successfully Sent, Duplicates Blocked, Failure Rate | Renders: StatCard.jsx, Recharts Campaign Visualizer, Recent Activity Table | +---> [ ContactsPage.jsx ] | Fetches: contactsService.getContacts(start, end, search) | State: searchTerm (debounced), activeContact (selected for modal inspection) | Renders: 1,842 Contacts Table, Search Input, Modal.jsx with full contact inspector | +---> [ CampaignPage.jsx ] | Form State: startRow, endRow, delaySeconds, isDryRun, isForce | Safety: Modal.jsx triggers interactive confirmation prior to dispatch | Dispatches: outreachService.sendOutreach() -> Displays AlertBanner with result summary | +---> [ PreviewPage.jsx ] | State: previewStart, previewEnd, previewList | Dispatches: outreachService.getPreview() -> Renders exact rendered email copies | Badging: Shows "ALREADY SENT" badge if recipient was previously emailed | +---> [ LogsPage.jsx ] Fetches: logsService.getLogs(limit, offset) Features: Chronological audit history, status badges, CSV download export button
+-------------------------------------------------------------------------------------------------+
```

Centralized Error Unwrapping Interceptor

In `src/services/api.js`, an Axios response interceptor unwraps all backend errors before they hit component code:

- **FastAPI Detail Strings:** If backend throws `HTTPException(400, detail='Error message')`, it extracts the string directly.
- **Pydantic Validation Arrays:** If input fails schema checks, it joins all field violation strings.
- **Connection Failures:** If FastAPI is stopped, it alerts: *'Cannot connect to backend server on 127.0.0.1:8000'*.

8. SQLite State Machine & Transactional Audit Trail Workflow

Persistence in Outreach-Hub is managed by an embedded SQLite database (`backend/data/email_sender.db`). The system acts as a state machine where every attempted outreach record transitions into one of four immutable states.

```
+-------------------------------------------------------------------------------------------------+ | SQLITE AUDIT
LOG STATE TRANSITIONS |
+-------------------------------------------------------------------------------------------------+ [ Outreach
Attempt Triggered ] | +-------------------------+-------------------------+ | | (dry_run == True) (dry_run == False) |
| v v [ STATUS: DRY_RUN ] Duplicate check in SQLite: Simulated run logged has_been_sent(email)? Zero emails sent / \
(YES)/ \(NO) v v [ STATUS: SKIPPED ] SMTP Send Initiated Reason: Duplicate / \ Zero emails sent (Success)/ \(Failure) v v
[ STATUS: SENT ] [ STATUS: FAILED ] Recorded in DB Error message logged Blocks future dups Can be retried
+-------------------------------------------------------------------------------------------------+
```

The Four Immutable Audit States

Status Code	Trigger Condition	Impact on Future Outreaches
SENT	Email successfully accepted by Gmail SMTP server (code 250).	**Blocks future sends.** Recipient will be SKIPPED in all future campaigns unless force: true is passed.
SKIPPED	Recipient email was found in SQLite with prior SENT status.	Audited with reason string. Zero emails sent. Does not affect prior SENT records.
DRY_RUN	Campaign executed in simulation mode (`dry_run: true`).	Audited for verification. **Does NOT block** future live sends (only SENT status blocks).
FAILED	SMTP rejected connection, invalid recipient, or network drop.	Full error string saved. **Does NOT block** future sends; can be retried immediately.

9. Headless CLI Execution Workflow & Operational Diagnostics

For power users, scripts, or automated environments, the entire platform can be driven directly via `backend/cli.py` without starting a web browser or running a frontend server.

Recommended Daily Outreach Operational Routine

- **Phase 1 (System Diagnostic):** Run python cli.py health to confirm SQLite connectivity, PDF presence (1,842 contacts), and Gmail App Password authentication.
- **Phase 2 (Inspect Targets):** Run python cli.py contacts --search "SourceFuse" or view rows --start 1 --end 10.
- **Phase 3 (Preview Personalization):** Run python cli.py preview --start 1 --end 5 to inspect the exact rendered subject and body text.
- **Phase 4 (Safe Simulation):** Run python cli.py send --start 1 --end 5 --dry-run. Verify the dry-run summary indicates zero errors.
- **Phase 5 (Live Dispatch):** Run python cli.py send --start 1 --end 5. Review the interactive warning and press y to confirm.
- **Phase 6 (Audit Review):** Run python cli.py logs --limit 10 to confirm all records are saved as SENT in SQLite.

Fast Diagnostic & Troubleshooting Cheat Sheet

- **SMTP Authentication Error (535 5.7.8):** Your Gmail account requires 2-Step Verification and a 16-character App Password (not your personal password). Set SMTP_PASSWORD in backend/.env.
- **Resume.pdf Missing Error:** The email dispatcher strictly requires backend/Resume.pdf to exist before dispatching. Ensure your resume PDF is located in the backend/ directory.
- **Port 8000 Already in Use:** A background FastAPI process is already running. Check running tasks or kill PID using Get-NetTCPConnection -LocalPort 8000 in PowerShell.
- **Recipient Marked SKIPPED:** Recipient has already received an email according to SQLite records. If you intentionally want to re-contact them, pass --force in CLI or toggle Force in the web UI.

Outreach-Hub Workflow & Architecture Engineering Manual
Engineered for high performance, deterministic behavior, and complete local privacy.
Built by **Vishal Yadav** (Full Stack Developer) • Jaipur, Rajasthan • github.com/vishal-1459